# Coding Assignment 2: Empirical Timing vs. Predicted Growth

Auqib Hamid Lone
*Department of Computer Science & Engineering, GCET Ganderbal*

August 24, 2026

**Due:** two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

## 2.1 Why this problem

Week 2 built a vocabulary — $T(n)$, $O$, $\Omega$, $\Theta$, best/worst/average case — and used it on three pieces of code: `linear_search`, `binary_search`, and a nested-loop duplicate check. Every claim in that lecture ("linear search is $O(n)$ worst case," "binary search is $O(\log n)$ worst case," "the nested loop is $\Theta(n^2)$") was a *prediction* derived on paper, never checked against a running program. This assignment closes that loop in two parts: first you *implement* the three algorithms yourself (Part 1, autograded for correctness); then you *time* your own implementations across growing input sizes and compare what you measure against what the lecture predicted (Part 2, written and instructor-graded). This is exactly the lecture's own closing exercise — "Before You Go: Predict" — except now you get to find out if your prediction was right.

## 2.2 Part 1: Implement the three functions

The contract is given to you in `analysis.h` — *do not modify it.* Implement the three functions in `analysis.c` (start from `starter_analysis.c`):

```
int linear_search(const int A[], int n, int key);
int binary_search(const int A[], int n, int key);
int has_duplicate(const int A[], int n);
```

**`linear_search`** scans `A[0..n-1]` from the front, comparing `A[i]` to `key`. Return the index of the *first* match, or $-1$ if `key` never appears. No precondition on the array's order.

**`binary_search`** assumes `A[0..n-1]` is *already sorted in ascending order* — this precondition is not checked by the function; calling it on an unsorted array is the exact anti-pattern the lecture warned against, and this assignment does not test that misuse. Use the lecture's own halving loop: `lo = 0`, `hi = n - 1`, `mid = lo + (hi - lo) / 2`, narrowing `[lo, hi]` to whichever half could still contain `key`. Return a matching index, or $-1$ if the interval becomes empty.

**`has_duplicate`** must use the *pairwise nested-loop* method from lecture: for every `i`, compare `A[i]` against every `A[j]` with `j > i`, returning 1 the instant a match is found and 0 if the loops finish without one. **Do not sort first and do not use a hash-based lookup** — the point of this problem is to produce, in your own code, the $\Theta(n^2)$ pattern from lecture, so that Part 2 has something real to measure. A cleverer duplicate check would defeat the purpose of the assignment.

### 2.2.1 Constraints

- $0 \leq n \leq 2000$ for correctness testing; your code must also run correctly on $n = 0$ (empty array) and $n = 1$.
- Worst-case bounds — `linear_search`: $O(n)$. `binary_search`: $O(\log n)$. `has_duplicate`: $O(n^2)$. Part 2 asks you to confirm these empirically.
- Your code must compile with `gcc -Wall -Wextra` without errors, with no global/static mutable state.

### 2.2.2 Worked examples (these are the visible tests)

- **`linear_search`** on `A = [42, 17, 99, 3, 56]` ($n = 5$):
  - `linear_search(A, 5, 3)` returns 3.
  - `linear_search(A, 5, 7)` returns $-1$ (absent).
- **`binary_search`** on the sorted array `A = [2, 9, 14, 23, 31, 40, 58]` ($n = 7$):
  - `binary_search(A, 7, 23)` returns 3.
  - `binary_search(A, 7, 15)` returns $-1$ (absent, falls between 14 and 23).
- **`has_duplicate`**:
  - on `A = [5, 8, 1, 8, 3]` returns 1 (8 repeats).
  - on `A = [5, 8, 1, 3]` returns 0.

### 2.2.3 What to submit for Part 1

1. `analysis.c` — your completed implementation (edit `starter_analysis.c`).
2. `analysis.h` — unchanged, but include it so the submission compiles standalone.

Sanity-check locally before submitting:

```
gcc -Wall -Wextra -o test_visible test_visible.c analysis.c
./test_visible
```

All worked examples above should pass. The autograder then compiles your `analysis.c` against a *hidden* test suite. It covers:

- boundary sizes (empty, single element);
- every position (front/middle/tail/absent) for the two searches;
- all-duplicate, no-duplicate, and duplicate-at-the-last-pair arrays for `has_duplicate`; and
- a batch of random inputs checked against a brute-force oracle.

## 2.3 Part 2: Time your own code

This part is **not** graded by the autograder — it is graded by the instructor from what you submit in `timing_report.md`.

Once `analysis.c` passes the visible tests, compile and run the provided `timing_harness.c` against it:

```
gcc -O2 -Wall -Wextra -o timing_harness timing_harness.c analysis.c
./timing_harness
```

**Do not use `-O0`** — at low optimization the constant-factor overhead of an unoptimized build can swamp the very signal you are trying to measure.

The harness runs each of your three functions in its *worst case* (the case the lecture derived a bound for: `key` absent for both searches, a no-duplicate array for `has_duplicate`) at several geometrically increasing values of $n$, and prints one timing table per function. It repeats each call enough times internally to get a measurement above your system clock's tick resolution — you do not need to do anything about this, just run it.

### 2.3.1 What to submit for Part 2

Copy `timing_report_template.md` to `timing_report.md`, paste in your three printed tables, and answer the three short questions there (2–4 sentences each):

1. For `linear_search`, what ratio does $O(n)$ predict when $n$ doubles? Is that roughly what you measured?
2. For `binary_search`, what ratio does $O(\log n)$ predict when $n$ doubles (hint: it is far from 2)? Compare to your table.
3. For `has_duplicate`, what ratio does $O(n^2)$ predict when $n$ doubles? Compare to your table, and contrast against Question 1 — this is the practical version of the lecture's "Numbers Side by Side" comparison.

## 2.4 What to submit, in total

1. `analysis.c`, `analysis.h` (Part 1 — autograded).
2. `timing_report.md` (Part 2 — instructor-graded).